FUNDACIÓN ARGENTINA DE SMALLTALK · 15/10/2026

protoST

El Smalltalk que conocen, desde un archivo,
en lugares donde hoy Smalltalk no llega.

Gustavo Marino

QUÉ ES ESTO

Un experimento que busca entusiastas.

protoST no es un Smalltalk mejor y no compite con el entorno vivo. Es otra cosa: el código que ya saben escribir, corriendo como un programa más, sobre un núcleo con hilos nativos y otros lenguajes al lado.

Todo lo que muestro está en el repositorio y se puede verificar.

PREGUNTAS, NO RESPUESTAS

¿Les suena?

Desplegar

¿Cuánto cuesta poner una imagen como servicio detrás de otra cosa?

Núcleos

¿Cuántos núcleos usa su código cuando la máquina tiene doce?

Integrar

¿Cómo conviven con Python, JavaScript, colas, bases?

Compartir

¿Cómo comparten código como texto con quien no usa una imagen?

POR QUÉ AHORA ES POSIBLE

protoCore: un núcleo, varios lenguajes

Inmutable

Colecciones que se "modifican" creando una versión nueva que comparte casi todo con la anterior.

Paralelo

Hilos nativos sin lock global y un recolector de basura concurrente.

Compartido

JavaScript, Python, Clojure, Scala y protoST sobre el mismo modelo de objetos.

LA IDEA QUE LO HACE SEGURO

Un objeto mutable es una referencia a una instantánea inmutable

`cuenta`

referencia atómica



```
{ owner: Ana balance: 120 }
```

instantánea: nadie la cambia

Un mensaje a otro hilo lleva una referencia, no una copia, y cada lectura ve una instantánea consistente: nunca un estado a medio escribir.

DEMO 1 · EL CÓDIGO QUE YA CONOCEN

```
Account >> withdraw: amount
amount > balance ifTrue: [
^ InsufficientFunds new signal: owner , ' has only ' , balance printString].
balance := balance - amount

[attempts := attempts + 1. ana withdraw: 130]
on: InsufficientFunds
do: [:e | ana deposit: 50. e retry].
```

Clases, **printOn:**, retorno no local, excepciones con **retry** y **resume:**, fracciones exactas. Un archivo de texto; una línea en blanco cierra un método.

ACTORES

Cualquier objeto puede ser un actor

```
c := Counter new asActor.
1 to: 1000 do: [:i | c increment].
c increment wait "⇒ 1001"
```

Cada mensaje devuelve un **Future** enseguida (salvo identidad, nil y printString, que responde el proxy).

El actor procesa un mensaje por vez: su estado son variables de instancia comunes, sin locks.

Los actores comparten un pool de hilos nativos.

DEMO 2 · PARALELISMO REAL

El mismo cálculo, en un hilo y en doce actores

```
futures := ranges collect: [:r |
PrimeCounter new asActor countFrom: r first to: r last].
parallel := futures inject: 0 into: [:sum :f | sum + f wait]
```

Los dos resultados deben coincidir (4946 primos); los tiempos se miden en vivo.

IDEAS, NO PROMESAS

Mundos que podría abrir

Gemelos digitales

Un actor por componente físico: cada evento se procesa atómicamente y los componentes conversan por mensajes.

Servicios

Un proceso que arranca en decenas de milisegundos, lee archivos y la entrada estándar, habla HTTP y TCP y corre otros programas: sin imagen que desplegar.

Otros runtimes

protoCore 2.6 ya permite que dos runtimes en un proceso usen los objetos del otro sin copiarlos. Falta que los binarios se carguen entre sí.

DEMO 4 · UN GEMELO CONECTADO

Otro programa entrega los datos; protoST los procesa y los publica

```
server := HTTPServer on: 0 handler: [:request |
(HTTPResponse ok: pump statusJson wait) contentType: 'application/json'].
log writeStreamDo: [:out |
Stdio stdin linesDo: [:line |
state := (pump reading: (JSON parse: line)) wait.
out nextPutAll: line; lf]].
```

python3 04-connected-twin.feed | protost 04-connected-twin.st: lecturas JSON por la entrada estándar, la bomba en un actor, su estado por HTTP y cada lectura en un archivo.

RENDIMIENTO, CON MÉTODO

Cada benchmark verifica su resultado

Medición	protoST	CPython
Arranque (una expresión)	32 ms	36 ms
Un hilo: el trabajo, sin el arranque (geomean)	16–27× más lento	1.00
Doce actores: pool contra 1 worker	2.5–2.8×	—
saturation_big: techo (4 workers)	≈ 2×	—

Fuente: benchmarks/reports/2026-10-11-protocore-2.14.5.md (main con protoCore 2.14.5; sintético, una notebook). De 5× (bucles) a 30× (despacho recursivo) y 80× (excepciones). No es rápido: es el mismo código en varios núcleos, sin locks.

LO QUE PROTOST NO ES

Todo escrito y verificado

— Sin imagen ni browser: el programa es el archivo

— Strings inmutables (WriteStream para construir)

— Metaclase delgada; sin thisContext

— Una línea en blanco cierra un método

— Símbolos cortos iguales a strings

— Un actor que espera no atiende otros mensajes

Tutorial, capítulo 14: qué difiere, por qué y qué escribir en su lugar.

CÓMO SE VERIFICÓ

Revisiones adversariales, antes de esta charla

≈1100 programas

Escritos como los escribe un programador Pharo; cada resultado silenciosamente incorrecto se corrigió con su test.

Una revisión de la E/S

3 defectos críticos y 8 importantes, corregidos antes de publicar; cada uno con un test que falló primero.

1100 tests

Conformidad, unidad, CLI, ejemplos y la documentación; en Linux, macOS y Windows.

Un cuelgue raro

Capturado con el proceso vivo: un despertar perdido en std::counting_semaphore de libstdc++ 13.

CÓMO PARTICIPAR

Escriban un programa como lo escribirían ustedes.

Y cuéntenme todo lo que les sorprenda.

`github.com/gamarino/protoST`

Paquetes para Ubuntu 24.04 y Windows (WSL2) en Releases · docs/TUTORIAL.md · capítulo 14

DEMO 3 · GEMELO DIGITAL (MÓDULO)

```
cycle 3 temp 72 flow 36 vib 3 running
cycle 4 temp 76 flow 48 vib 4 running
temperature above 75: stopped
time per cycle: 56 ms (three 50 ms reads each)
```

Una bomba como máquina de estados en un actor; tres sensores con 50 ms de E/S simulada leídos en paralelo en cada ciclo; una alarma detiene la bomba.

MÓDULO · PROTOCORE POR DENTRO

Celdas, instantáneas y un GC que se adelanta

Celdas de 64 bytes

Del tamaño de una línea de caché; enteros pequeños dentro del puntero.

Listas y cuerdas

Árboles AVL y ropes inmutables: cambiar cuesta O(log n) nodos nuevos.

GC concurrente

Arranca cada ciclo antes de llegar al techo y barre con varios cursores e hilos auxiliares.

MÓDULO · PREGUNTAS ABIERTAS

¿Qué les sería útil primero?

- Que los runtimes se carguen entre sí (el núcleo ya lo permite)
- thisContext y herramientas de depuración
- Rendimiento del envío de mensajes
- Más protocolo de la biblioteca de clases; WebSockets y HTTP/2